手写作业自动批改系统 - 单元测试计划文档

1. 引言

1.1 目的

本计划旨在指导开发团队对“手写作业自动批改系统”的核心代码单元进行系统性测试。重点验证业务逻辑的正确性、算法的准确性（特别是图像预处理和评分逻辑）、异常处理的健壮性以及微服务间接口调用的可靠性。

1.2 测试范围

- **Spring Boot 后端**: Service 层业务逻辑、Util 工具类、Entity 数据转换。
- **Flask OCR 微服务**: 图像预处理函数、OCR 结果分配逻辑、API 接口参数校验。
- 排除范围
  :
  - 前端 UI 交互测试。
  - 全链路压力/性能测试（属于集成/性能测试范畴）。
  - 真实的 PaddleOCR 模型训练效果验证（依赖外部模型，单元测试使用 Mock）。

1.3 测试工具

- **Java**: JUnit 5, Mockito, AssertJ, Spring Boot Test.
- **Python**: pytest, unittest, pytest-mock, numpy testing utilities.
- **覆盖率工具**: JaCoCo (Java), coverage.py (Python).

---

2. 测试策略

2.1 mocking 策略

由于系统依赖外部资源（数据库、文件系统、PaddleOCR 引擎、HTTP 远程调用），单元测试必须广泛使用 Mock：

- **数据库**: Mock Mapper 接口，不连接真实 MySQL。
- **文件系统**: 使用 `java.nio.file.Files.createTempDirectory` 或 Python `tempfile`，测试后自动清理。
- OCR 引擎:
  - Java 端：Mock `OcrService` 的 HTTP 调用返回固定 JSON。
  - Python 端：在 `get_ocr_engine` 中强制注入 `MockOcrEngine`，或使用 `unittest.mock.patch` 替换 PaddleOCR 类。
- **时间**: Mock `LocalDateTime.now()` 或 `time.time()` 以确保测试结果可复现。

2.2 数据驱动测试

对于评分逻辑和图像参数，采用数据驱动方式（JUnit `@MethodSource` / Python `@pytest.mark.parametrize`），覆盖多种边界情况。

2.3 覆盖率目标

- **行覆盖率 (Line Coverage)**: ≥ 85%
- **分支覆盖率 (Branch Coverage)**: ≥ 80%
- **核心算法模块 (Grading, ImageProcessor)**: ≥ 95%

---

3. 详细测试用例设计

3.1 Spring Boot 后端模块

3.1.1 认证服务 (`AuthService`)

表格

用例 ID	测试场景	输入数据	预期结果	关键断言
UT-AUTH-01	注册成功	唯一用户名，合法密码	返回 LoginResponse，DB 插入调用 1 次	密码已加密 (BCrypt)，Role 正确
UT-AUTH-02	注册用户名冲突	已存在的用户名	抛出 `RuntimeException("用户名已存在")`	Mapper `selectCount` 返回 >0
UT-AUTH-03	登录成功	正确用户名/密码	返回包含 JWT 的 Response	Token 非空，包含 userId/role
UT-AUTH-04	登录密码错误	正确用户名，错误密码	抛出 `RuntimeException("密码错误")`	`passwordEncoder.matches` 返回 false
UT-AUTH-05	登录用户禁用	状态=0 的用户	抛出 `RuntimeException("用户不存在或已被禁用")`	查询条件包含 `status=1`

3.1.2 批改引擎服务 (`GradingService`)

核心逻辑，需高覆盖率

表格

用例 ID	测试场景	输入数据 (Student Answer / Key)	预期结果	关键断言
UT-GRD-01	精确匹配 - 完全一致	"A" / "A"	得分=满分，isCorrect=1	忽略大小写/空格后相等
UT-GRD-02	精确匹配 - 格式差异	" a " / "A"	得分=满分，isCorrect=1	`normalizeAnswer` 生效
UT-GRD-03	精确匹配 - 错误	"B" / "A"	得分=0，isCorrect=0	Feedback 包含参考答案
UT-GRD-04	关键词匹配 - 全中	"重力加速度向下" / ["重力", "向下"]	得分=满分，isCorrect=1	命中数=2，总数=2
UT-GRD-05	关键词匹配 - 部分中	"只有重力" / ["重力", "向下"]	得分=50%，isCorrect=2	命中数=1，计算比例
UT-GRD-06	关键词匹配 - 未中	"摩擦力" / ["重力", "向下"]	得分=0，isCorrect=0	命中数=0
UT-GRD-07	人工审核模式	任意 / 任意	得分=0，gradingType=2	跳过自动逻辑，标记人工
UT-GRD-08	空答案处理	"" / "A"	得分=0，不抛异常	鲁棒性测试
UT-GRD-09	总分计算精度	多题得分 (33.333 + 33.333)	总分保留 2 位小数 (66.67)	`RoundingMode.HALF_UP`

3.1.3 提交服务 (`SubmissionService`)

表格

用例 ID	测试场景	前置条件	预期结果	关键断言
UT-SUB-01	提交成功	未提交过	返回 Submission 对象，状态=0	文件保存路径正确，异步方法被调用

用例 ID	测试场景	前置条件	预期结果	关键断言
UT-SUB-02	重复提交拦截	数据库中已有记录	抛出 `RuntimeException`	不调用保存文件逻辑
UT-SUB-03	异步处理成功流程	Mock OCR 返回结果	状态流转 0->1->2->4	`GradingService.gradeSubmission` 被调用
UT-SUB-04	异步处理异常捕获	Mock OCR 抛异常	状态流转至 5 (失败)	日志记录错误，事务不回滚主提交记录
UT-SUB-05	文件名生成规则	相同学生同天多次	文件名不同 (UUID)	防止文件覆盖

3.1.4 OCR 服务调用 (`OcrService`)

表格

用例 ID	测试场景	Mock 响应	预期结果	关键断言
UT-OCR-01	解析正常响应	Code 200, 含 results 数组	返回 Map<Integer, String>	题号与文本映射正确
UT-OCR-02	解析错误响应	Code 500 或 JSON 格式错	返回空 Map	不抛异常，记录 Warn 日志
UT-OCR-03	网络超时/不可达	抛 ConnectException	返回空 Map	降级处理，不阻断业务

3.2 Flask OCR 微服务模块

3.2.1 图像预处理 (`image_processor.py`)

需构造特定的 Numpy 数组或测试图片

表格

用例 ID	测试场景	输入数据特征	预期结果	关键断言
UT-IMG-01	正常图片预处理	标准 RGB 图片字节	返回 BGR Numpy 数组	Shape 一致，类型 uint8

用例 ID	测试场景	输入数据特征	预期结果	关键断言
UT-IMG-02	损坏图片数据	随机字节流	抛出 `ValueError`	`cv2.imdecode` 返回 None 时处理正确
UT-IMG-03	倾斜校正 - 小角度	模拟倾斜 10° 的二值图	返回旋转后的图像	角度计算逻辑覆盖
UT-IMG-04	倾斜校正 - 大角度	模拟旋转 90° 的图	返回原图 (不处理)	`abs(angle) > 15` 分支覆盖
UT-IMG-05	空白/噪点图	全白或极少像素	返回原图或处理后图	`len(coords) < 10` 分支覆盖

3.2.2 识别结果分配 (`ocr_engine.py`)

表格

用例 ID	测试场景	输入 (Texts, Regions)	预期结果	关键断言
UT-REC-01	自动分题 - 顺序	按 Y 轴排序的 3 个文本块	{1: text1, 2: text2, 3: text3}	排序逻辑正确
UT-REC-02	区域分配 - 完全匹配	指定 Y 区间包含对应文本	题号与区域内文本绑定	`center_y` 计算及区间判断
UT-REC-03	区域分配 - 跨区/无匹配	文本不在任何区域	对应题号为空或忽略	边界条件测试
UT-REC-04	空识别结果	OCR 返回 []	返回空 Dict	空指针保护
UT-REC-05	Mock 引擎调用	无 PaddleOCR 环境	返回 "模拟识别结果"	`MockOcrEngine` 生效

3.2.3 API 接口 (`app.py`)

表格

用例 ID	测试场景	请求内容	预期 HTTP 响应	关键断言
UT-API-01	健康检查	GET /api/health	200 OK, code=200	服务存活
UT-API-02	识别 - 缺少文件	POST 无 image 字段	400 Bad Request	错误消息明确

用例 ID	测试场景	请求内容	预期 HTTP 响应	关键断言
UT-API-03	识别 - 空文件名	POST image 字段名为空	400 Bad Request	校验逻辑
UT-API-04	识别 - regions 解析失败	JSON 格式错误	200 OK (降级为自动分题)	记录 Warning 日志，不报错
UT-API-05	识别 - 内部异常	Mock 预处理抛错	500 Internal Error	返回统一错误格式

---

4. 测试执行流程

1. 环境准备

   :

   - Java: 确保 Maven/Gradle 依赖下载完成 (`mvn test-compile`)。
   - Python: 创建虚拟环境，安装 `pytest`, `pytest-mock`, `opencv-python-headless` (CI 环境无需 GUI)。
2. 本地执行

   :

   - Java: `mvn test` 或 IDE 直接运行 Test Suite。
   - Python: `pytest tests/ --cov=.`
3. CI 流水线集成

   :

   - 配置 GitHub Actions / GitLab CI，每次 Push 自动运行单元测试。
   - 设置质量门禁：覆盖率低于 80% 或有任何 Case 失败则构建失败。
4. 缺陷修复与回归

   :

   - 发现 Bug -> 编写复现该 Bug 的测试用例 (Fail) -> 修复代码 -> 测试通过 (Pass)。

---

5. 交付物

1. 测试代码库

   :

   - `src/test/java/com/homework/...`
   - `tests/test_image_processor.py`, `tests/test_ocr_engine.py` 等。
2. 测试报告

   :

   - HTML 格式的覆盖率报告 (JaCoCo/Coverage.py)。
   - 单元测试执行结果摘要（通过/失败/跳过数量）。
3. 问题清单

：

- 测试过程中发现的逻辑漏洞、边界条件缺失列表及修复状态。

---

6. 风险评估与应对

风险点	影响	应对措施
OpenCV 环境依赖	Python 测试在 CI 环境因缺少 GL 库失败	使用 `opencv-python-headless` 替代完整版；在 Docker 中预装 `libgl1`。
时间相关逻辑	异步线程中时间断言失败	严格 Mock `LocalDateTime` 和 `time.time`，不依赖真实系统时间。
文件 IO 竞争	并行测试时文件路径冲突	每个测试用例使用独立的 `TempDir`，测试结束后强制清理。
异步测试难点	`@Async` 方法难以断言执行结果	使用 `CountDownLatch` 或在测试配置中将异步执行器改为同步执行器 (`SyncTaskExecutor`) 以便调试。